

TALLER PRACTICO: IMPLEMENTACION Y NANALISIS ASINTOTICO DE ALGORITMOS DE ORDENAMIENTO

CAMPO	DETALLE
Conjunto de datos	Base de datos SQLite-tabla personas
Total Registros	585 personas
Campo de ordenamiento	Puntaje_evaluacion
Lenguaje	Python

INTRODUCCION

Este informe documenta los resultados obtenidos en el taller práctico de algoritmos de ordenamiento , se implementaron tres métodos : Bubble sort , Inserction sort, Selection sort, aplicado sobre un conjunto de 585 registros de personas almacenada en una base de datos SQLite , para cada algoritmo se midio el tiempo de ejecución de forma precisa utilizando **time.perf_counter()**, excluyendo el tiempo de lectura de la base de datos

DESCRIPCION DE ALGORITMOS

Bubble sort : recorre el arreglo comparando elementos cercanos e intercambiándolos cuando están en el orden incorrecto . Cada recorrido deja el elemento mayoren su posición final , se aplica una bandera de optimización que detiene el proceso si no ocurrió ningún intercambio , mejorando el caso a complejidad lineal

Inserction sort : toma cada elemento y lo inserta dentro de la posición correcta dentro de la porción ya ordenada del arreglo , desplazando los elementos mayores hacia la derecha es especialmente eficiente con arreglos parcialmente ordenados o de un tamaño pequeño

Selection sort : en cada iteración busca el elemento mínimo en el subarreglo no ordenado y lo intercambio con el primero de ese subarreglo . A diferencia de los dos anteriores su complejidad no mejora en el mejor caso por que siempre realiza el mismo numero de comparaciones

TABLA COMPARATIVA DE TIEMPOS Y COMPLEJIDAD

ALGORITMO	TIEMPO (MS)	TIEMPO (S)	PEOR CASO	CASO PROMEDIO	MEJORCASO
Insertion sort	8.6736	0.008674	$O(n^2)$	$\theta(n^2)$	$\Omega(n)$
Selection sort	14.8114	0.014811	$O(n^2)$	$\theta(n^2)$	$\Omega(n^2)$
Bubble sort	19.8263	0.019826	$O(n^2)$	$\theta(n^2)$	$\Omega(n)$

ANALISIS DE COMPLEJIDAD ALGORITMICA

CASO	BUBBLE SORT	INSERTION SORT	SELECTION SORT
Peor caso (big O)	$O(n^2)$ arreglo invertido	$O(n^2)$ arreglo invertido	$O(n^2)$ siempre igual
Caso promedio (big θ)	$O(n^2)$ Aleatorio	$O(n^2)$ Aleatorio	$O(n^2)$ Aleatorio
Mejor caso (big Ω)	$\Omega(n)$ ya ordenado	$\Omega(n)$ ya ordenado	$\Omega(n^2)$ sin mejora

CONCLUSIONES

Los tres algoritmos tiene la misma complejidad cuadrática $O(n^2)$ en el peor caso y en el caso promedio , pero los tiempos empíricos muestran diferencias considerables que tienen una explicación mas precisa en su funcionamiento interno

En conclusión, aunque la notación asintótica los clasifica a los tres en $O(n^2)$, las constantes ocultas y el comportamiento ante datos reales generan diferencias medibles. Para conjuntos pequeños o parcialmente ordenados, Insertion Sort es la mejor opción entre los tres. Para cualquier caso que requiera alta eficiencia, se deben considerar algoritmos de orden $O(n \log n)$ como Merge Sort o Quick Sort.